

Static Code Analysis Lab

SAST & SCA

Francesca Craievich | IN2300026

Lab 07 | [505MI] Cybersecurity Lab | A.Y. 2025/2026

Tools & Repositories

Tools:

- SAST: SonarCloud (Online)
- SCA: Trivy by Aqua Security (CLI)
- SCA: OWASP Dependency Check v13 (Docker)

SonarCloud - analyzes source code for vulnerabilities, bugs, and code smells without executing the program

Trivy - scans project dependencies (lock files) for known CVEs in third-party libraries

Repositories:

VULNERABLE

Cacti

PHP - 714k LOC

github.com/Cacti/cacti

SECURE

service-points-location-capacity

Python - 2.2k LOC

github.com/francescacravich/service-points-location-capacity

Cacti Repository

Description:

- Open-source network monitoring and graphing platform built on RRDtool
- Scalable framework to collect, store, and visualize time-series data from network devices, servers, and applications

Why Selected:

- Known issues: large real-world codebase with documented security flaws, serves as a benchmark for SAST/SCA detection
- Multiple languages: PHP backend with JavaScript frontend (yarn + composer), enabling both SAST and SCA analysis
- Expectation: high detection rates and critical severity alerts

service-points-location-capacity

Description:

- Implementation of the mixed-integer linear programming (MILP) model from Raviv, T. (2023)
- Optimization model for locating and sizing automatic parcel lockers (APLs) in urban delivery networks

Why Selected:

- Baseline comparison: verifies how SAST/SCA tools handle a clean, well-structured codebase
- Personal project: full knowledge of the code allows accurate evaluation of tool findings
- Expectation: clean results with minimal findings

Cacti - SAST

✓

Quality gate: [Sonar way](#) ⓘ

Passed 

Security 31 Open issues E	Reliability 4.3k Open issues E	Maintainability 24k Open issues A
Accepted Issues 0 Valid issues that were not fixed 	Coverage A few extra steps are needed for SonarQube Cloud to analyze your code coverage. Set up coverage analysis 	Duplications 36.8% No conditions set on 977k Lines 

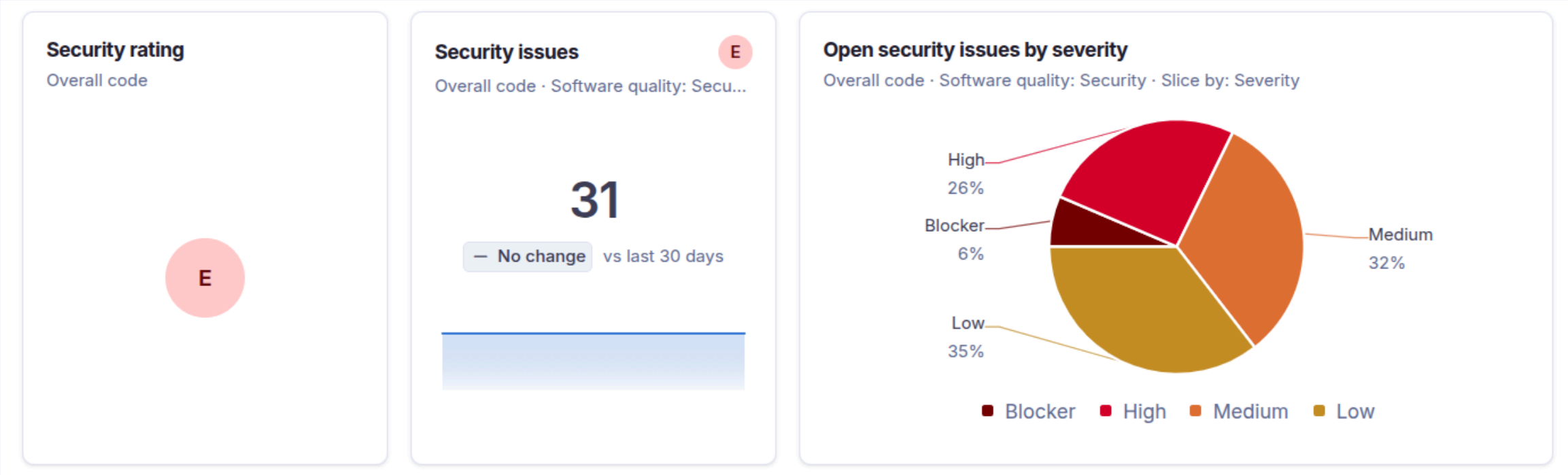
SonarCloud Findings:

- **Security: E - 31 issues**
- **Reliability: E - 4.3k issues**
- Maintainability: A - 24k code smells
- 714k LOC, 36.8% code duplication

Insight:

The project "Passed" the Quality Gate despite having E ratings in Security and Reliability. This is because the Quality Gate often applies only to new code, leaving legacy vulnerabilities hidden unless manually inspected.

Cacti - Security Issues



31 Security Issues

- 2 Blocker
- 8 High
- 10 Medium
- 11 Low

include/layout.js

Change this code to not perform client-side redirection based on user-controlled data.

Intentionality

Security

Blocker

cwe

+

Open

Not assigned

L4073

30min effort

1 year ago

lib/auth.php

Make sure this bcrypt password hash gets revoked, changed, and removed from the code.

Responsibility

Security

Blocker

cwe

secret

+

Open

Not assigned

L4299

30min effort

16 days ago

docker/Dockerfile.test

Copying recursively might inadvertently add sensitive data to the container. Make sure it is safe here.

Intentionality

Security

High

cwe

dockerfile

...

+

Open

Not assigned

L14

20min effort

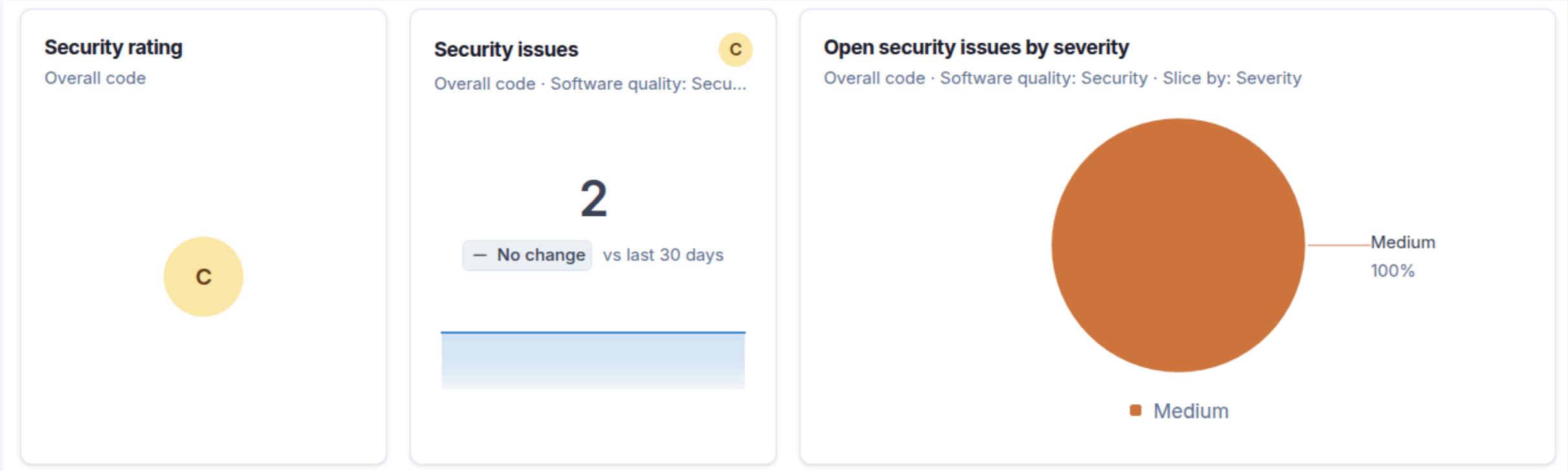
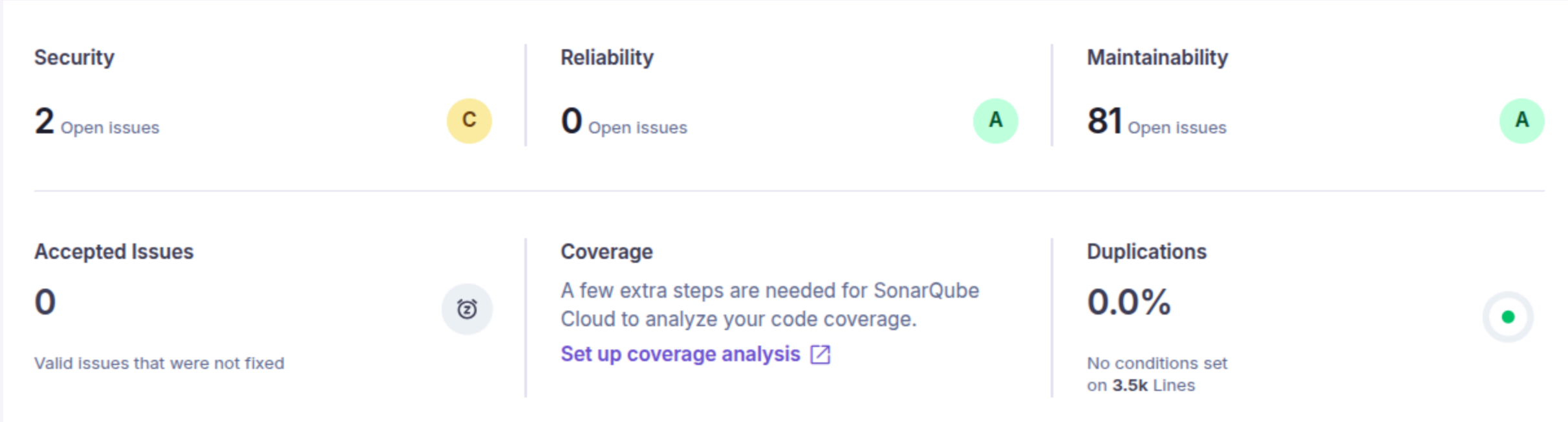
15 days ago

Key Finding:

Bcrypt password hash exposed in server responses (Blocker severity).

Combined with client-side open redirect vulnerabilities, this creates a critical attack surface for credential theft and phishing.

service-points-location-capacity - SAST



SonarCloud Findings:

- Security: C - 2 issues
- Reliability: A - 0 issues
- Maintainability: A - 81 code smells
- Duplications: 0.0%

Analysis:

Both security findings are false positives (next slide).

The 81 code smells are minor improvements: naming conventions and cognitive complexity refactoring.

2.2k Lines of Code / 0% duplication / well-structured codebase

SAST - False Positive Analysis

Where is the issue?	Why is this an issue?	How can I fix it?	Activity	More info
141 142 143 144 145 146 147 148	<pre># According to Raviv (2023), SP candidates are selected from ALL grid points # So we randomly sample from all grid points for both demand and SP # For demand points: use the first n_demand points (or random sample) if len(all_grid_points) > target_demand: # Randomly sample demand points demand_indices = random.sample(range(len(all_grid_points)), n_demand)</pre>			
149 150 151 152 153 154 155 156 157 158	<pre> demand_points = [all_grid_points[i] for i in demand_indices] demand_rates = [all_demand_rates[i] for i in demand_indices] else: demand_points = all_grid_points[:n_demand] demand_rates = all_demand_rates[:n_demand] # For SP locations: according to paper, sample from ALL grid points if len(all_grid_points) > target_sp: # Randomly sample SP locations from all grid points sp_indices = random.sample(range(len(all_grid_points)), n_sp)</pre>			

Why False Positive?

SonarCloud flagged `random.sample()` as an insecure Pseudo-Random Number Generator (PRNG).

But the function is used for geographic coordinate sampling, not cryptography.

Lesson:
SAST tools cannot understand business logic context.

SCA Results - Trivy

Cacti

Report Summary

Target	Type	Vulnerabilities
composer.lock	composer	0
include/vendor/lipis/flag-icons/yarn.lock	yarn	2

Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)

include/vendor/lipis/flag-icons/yarn.lock (yarn)
=====

Total: 2 (UNKNOWN: 0, LOW: 0, MEDIUM: 1, HIGH: 1, CRITICAL: 0)

Library	Vulnerability	Severity	Status	Installed Version	Fixed Version	Title
picomatch	CVE-2026-33671	HIGH	fixed	2.3.1	4.0.4, 3.0.2, 2.3.2	picomatch: Picomatch: Regular Expression Denial of Service via crafted extglob patterns https://avd.aquasec.com/nvd/cve-2026-33671
	CVE-2026-33672	MEDIUM				picomatch: Picomatch: Data integrity compromised via method injection with crafted POSIX bracket... https://avd.aquasec.com/nvd/cve-2026-33672

2 CVEs Found

picomatch (yarn.lock)

CVE-2026-33671
HIGH - ReDoS via
crafted extglob patterns

CVE-2026-33672
MEDIUM - Data integrity
compromised via POSIX
bracket injection

service-points-location-capacity

Report Summary

Target	Type	Vulnerabilities
-	-	-

0 vulnerabilities found

No dependencies with known CVEs

SCA Results - OWASP Dependency Check

Cacti

Project: Cacti

Scan Information ([show all](#)):

- *dependency-check version:* 13.0.0
- *Report Generated On:* Tue, 1 Sep 2026 08:41:29 GMT
- *Dependencies Scanned:* 350 (346 unique)
- *Vulnerable Dependencies:* 71
- *Vulnerabilities Found:* 146
- *Vulnerabilities Suppressed:* 0
- ...

Summary

Summary of Vulnerable Dependencies([click to show all](#))

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
additional-methods.js		pkg:javascript/jquery-validation@1.19.5	MEDIUM	2		3
additional-methods.min.js		pkg:javascript/jquery-validation@1.19.5	MEDIUM	2		3
follow-redirects:1.15.6		pkg:npm/follow-redirects@1.15.6	MODERATE	1		3
immutable:5.0.3		pkg:npm/immutable@5.0.3	CRITICAL	3		3
lodash:4.17.21		pkg:npm/lodash@4.17.21	HIGH	3		3
messages_ar.min.js		pkg:javascript/jquery-validation@1.19.5	MEDIUM	2		3
messages_az.min.js		pkg:javascript/jquery-validation@1.19.5	MEDIUM	2		3
messages_bg.min.js		pkg:javascript/jquery-validation@1.19.5	MEDIUM	2		3
messages_bn_BD.min.js		pkg:javascript/jquery-validation@1.19.5	MEDIUM	2		3

146 CVEs Found

- 350 dependencies scanned, 71 vulnerable
- CRITICAL: immutable
- HIGH: lodash

Comparison: Trivy vs OWASP DC

- Trivy scans only lock files (yarn.lock): found 2 CVEs
- OWASP DC also scans .js files: found 146 CVEs
- More data sources (NVD, RetireJS) = wider coverage

service-points-location-capacity

0 dependencies scanned, 0 vulnerabilities